This way, the configuration factory doesn't try to load application.{conf,json, properties}, but rather myapp.{conf,json,properties}. (This is required if you need multiple configurations in a single JVM.)

Another option is to use Java system properties. Sometimes this is the easiest approach because you can create a Bash script to run the application while setting a Java property pointing to the configuration of your choice instead of exploding JARs (that is, instead of providing the unzipped JAR structure with only the specific configuration for that environment).

System properties can be used for the name of the configuration file when you're using the `load` method without arguments—that is, `ConfigFactory.load()`. Setting system properties is as easy as adding the `-D` option. For example:

```
-Dconfig.file="config/myapp.conf"
```

when using the configuration file config/myapp.conf. When you use one of these properties, the default behavior of searching for the different .conf, .json, and .properties files is skipped.

You can use the following three system properties to control which configuration file the application reads:

- `config.resource` specifies a resource name, not a base name: for example, application.conf, not `application`.
- `config.file` specifies a file system path; again, it should include the extension.
- `config.url` specifies a URL.